2. REDIS Architecture

Wednesday, 23 September 2026 1:08 PM

we might have faced these questions in interviews or heard from someone:

- **Why Redis is so fast?**
- **Even though Redis is Single Threaded yet it is so fast, How?**

And most of the time, answer I see is - Because Redis stores data in *RAM*, that's why it is fast.

But in my view, this is not a complete answer.

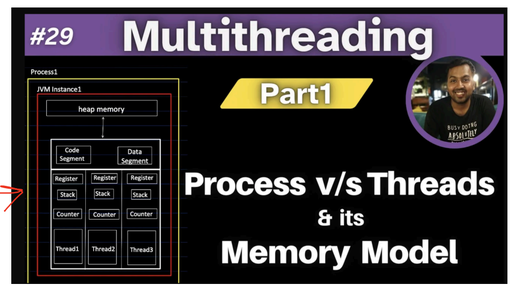
Redis is fast because of:

- **I/O Multiplexing**
- **Event Loop**
- **RESP protocol**
- **RAM storage**
- **I/O Threads**
- **fork() for persistence**

Lets go 1 level more deeper, to understand the Redis Architecture better:

Difference between Process and Thread

In Java Series, we have already covered what the difference between Process and Threads

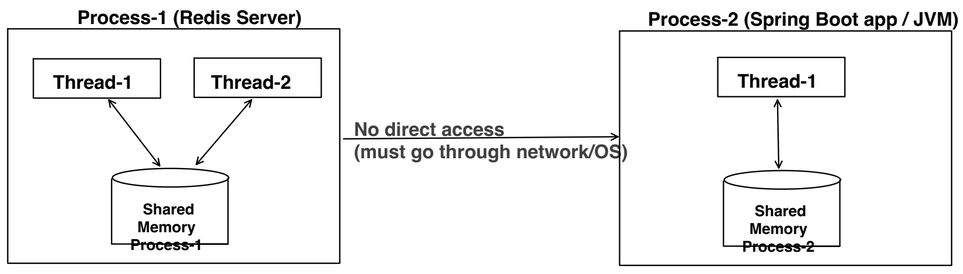


Process

- **Consider process as a sealed apartment, it get its own private memory space, its own set of resources.**
- **And nothing from outside can read or write directly into it without going through proper door (OS provided mechanisms like shared-memory, sockets, etc.)**

Threads

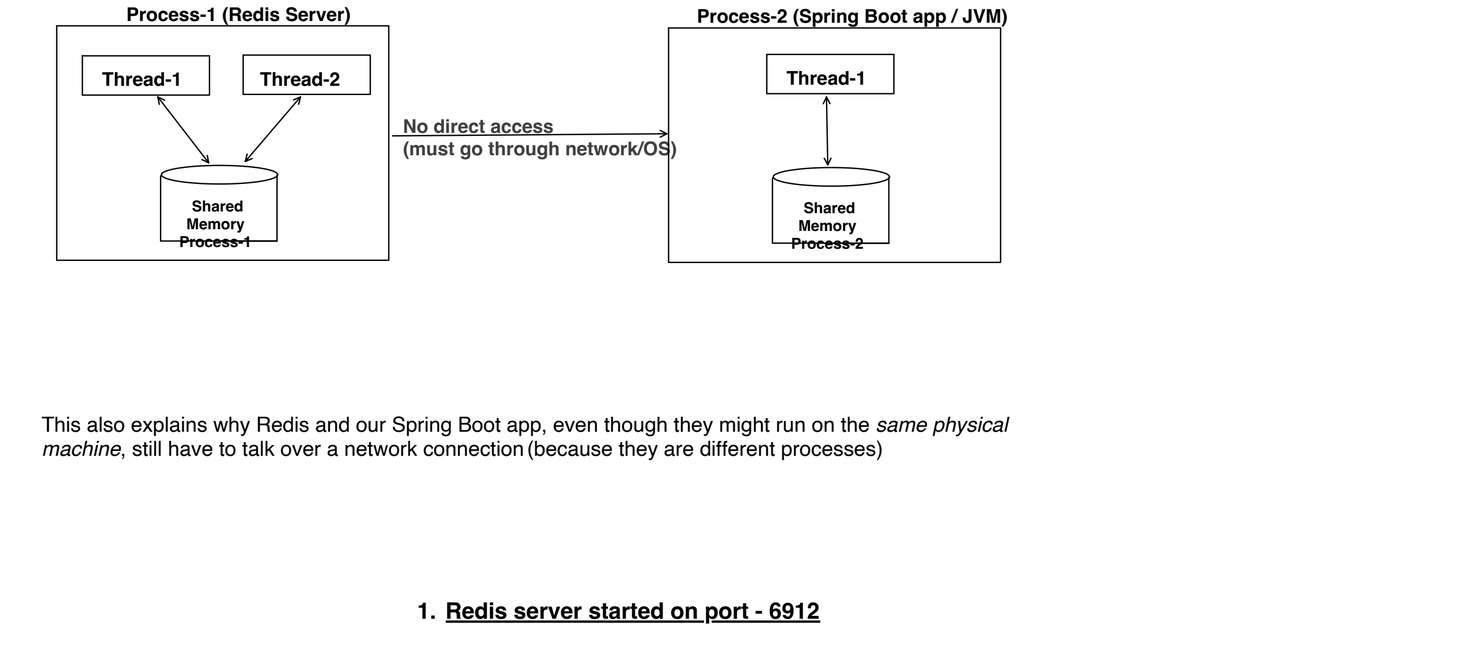
- **Thread lives inside process and run code.**
- **All threads within the same process share its process memory.**

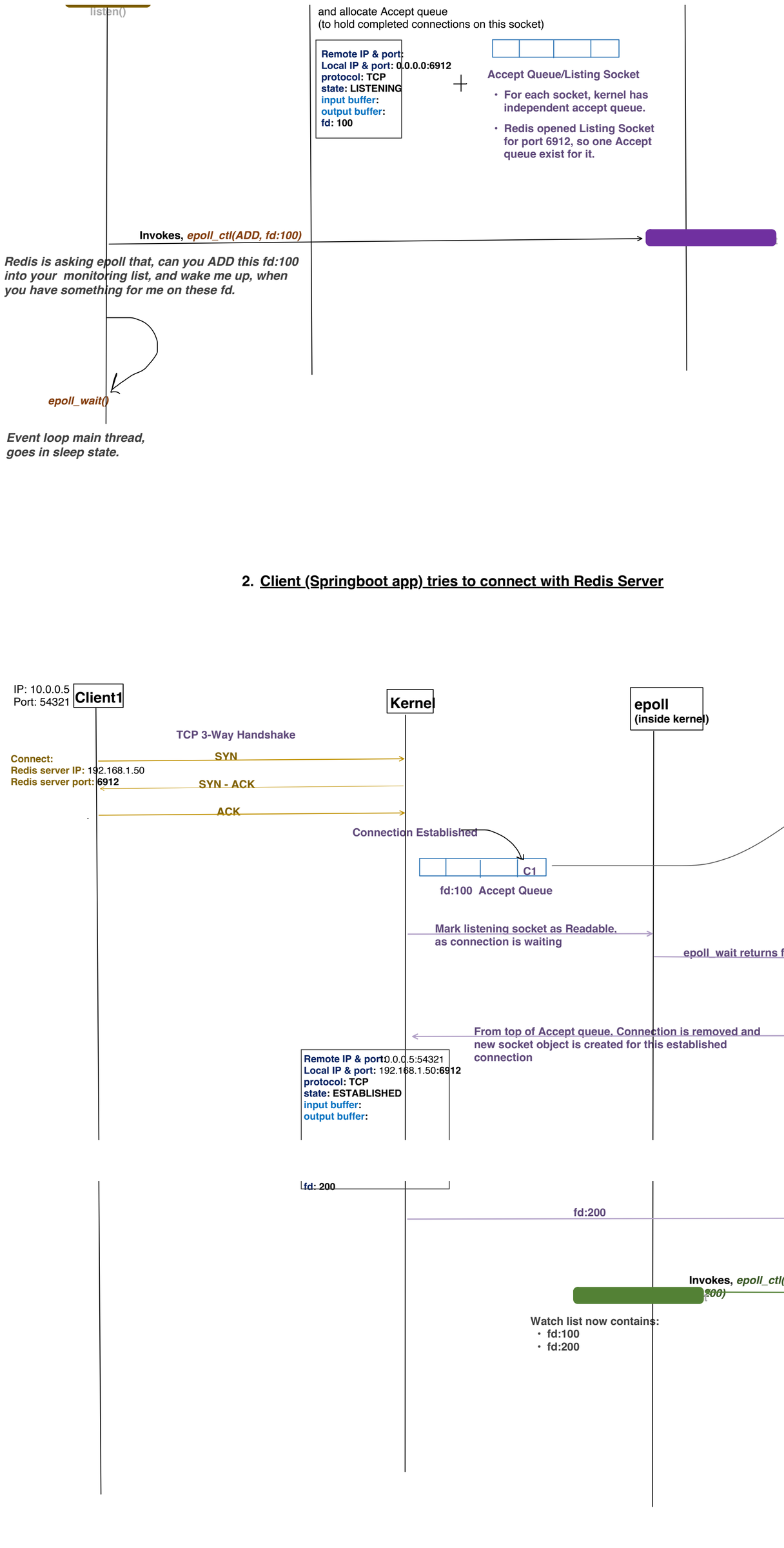


This also explains why Redis and our Spring Boot app, even though they might run on the *same physical machine*, still have to talk over a network connection (because they are different processes)

1. **Redis server started on port - 6912**

Redis is Single Threaded, and the thread which referred here is the one which run *EVENT LOOP* logic.
This main thread only does processing of the commands.
Since there is only 1 thread which does processing, that's why REDIS operations are called ATOMIC (guarantee that no 2 commands run
at the exact same time)





2. Client (Springboot app) tries to connect with Redis Server

3. Client (Springboot app) prepares to send the data

RESP(Redis Serialization Protocol)

- Its an encoding, which is very easy to parse.

'*' tells how many elements follow
'\r\n' its just a line ending marker (helps to tell parser that, where one piece ends and the next begins)

*3\r\n : 3 elements will follow
$3\r\n
SET\r\n
$3\r\n
name\r\n
$3\r\n
Shrayansh\r\n

'$' tells it's a Bulk String and how many bytes

*3\r\n : 3 elements will follow
$3\r\n : String with 3 bytes follow
SET\r\n : ← SET is a 3 bytes string
$4\r\n :
name\r\n :
$9\r\n :
Shrayansh\r\n :

Similarly

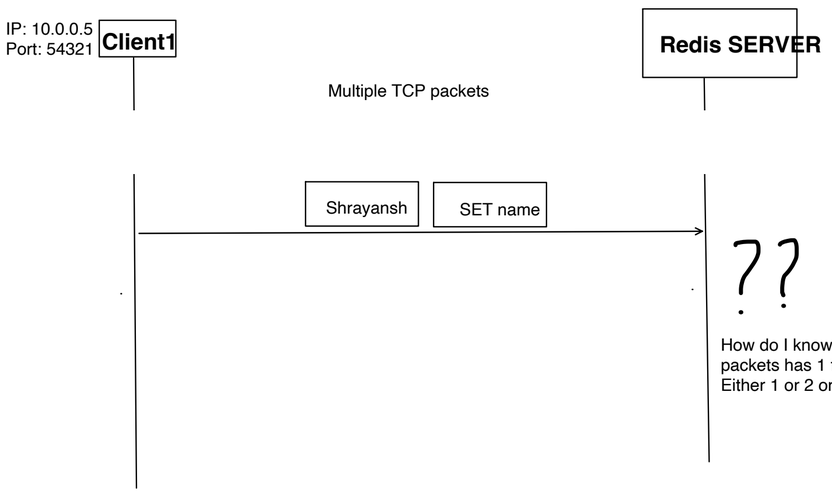
*3\r\n : 3 elements will follow
$3\r\n : String with 3 bytes follow
SET\r\n : SET 3 bytes string
$4\r\n : String with 4 bytes follow
name\r\n : name 4 bytes string
$9\r\n : String with 9 bytes follow
Shrayansh\r\n : Shrayansh 9 bytes string

Now you might be thinking, why we need this Encoding? Whats the use?

Lets say without RESP encoding, what problem can come

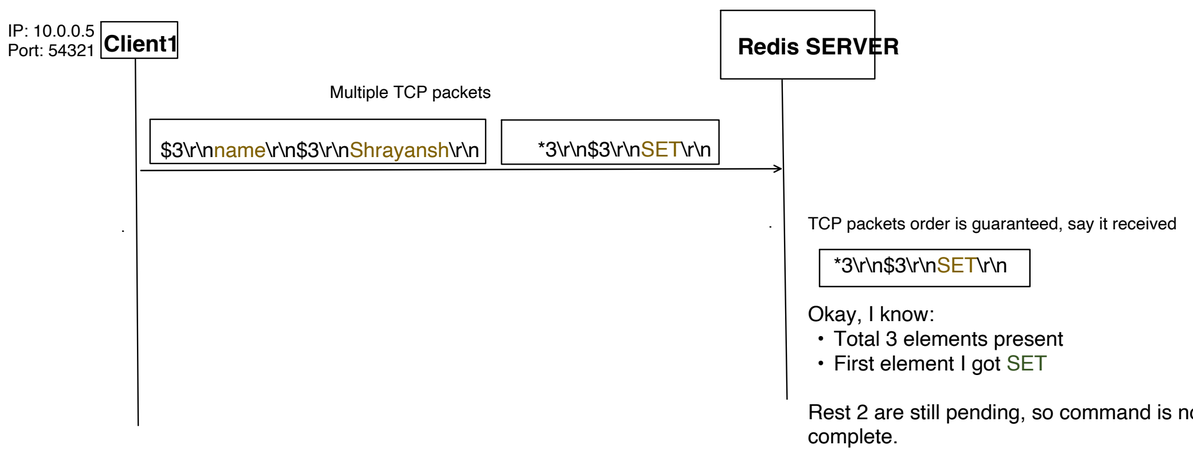
Client need to send below command:

SET name Shrayansh

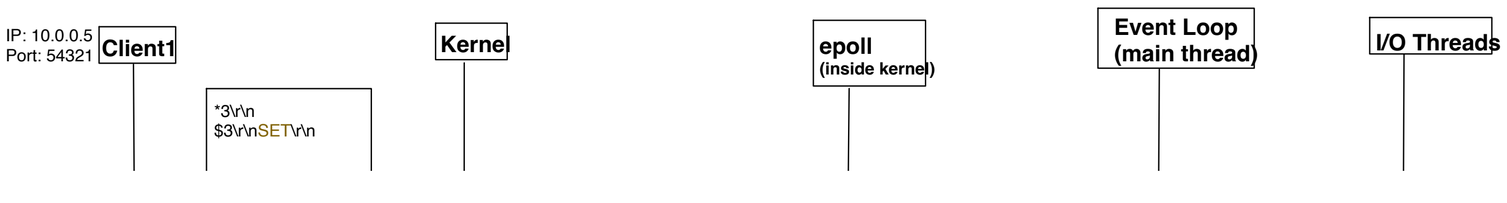


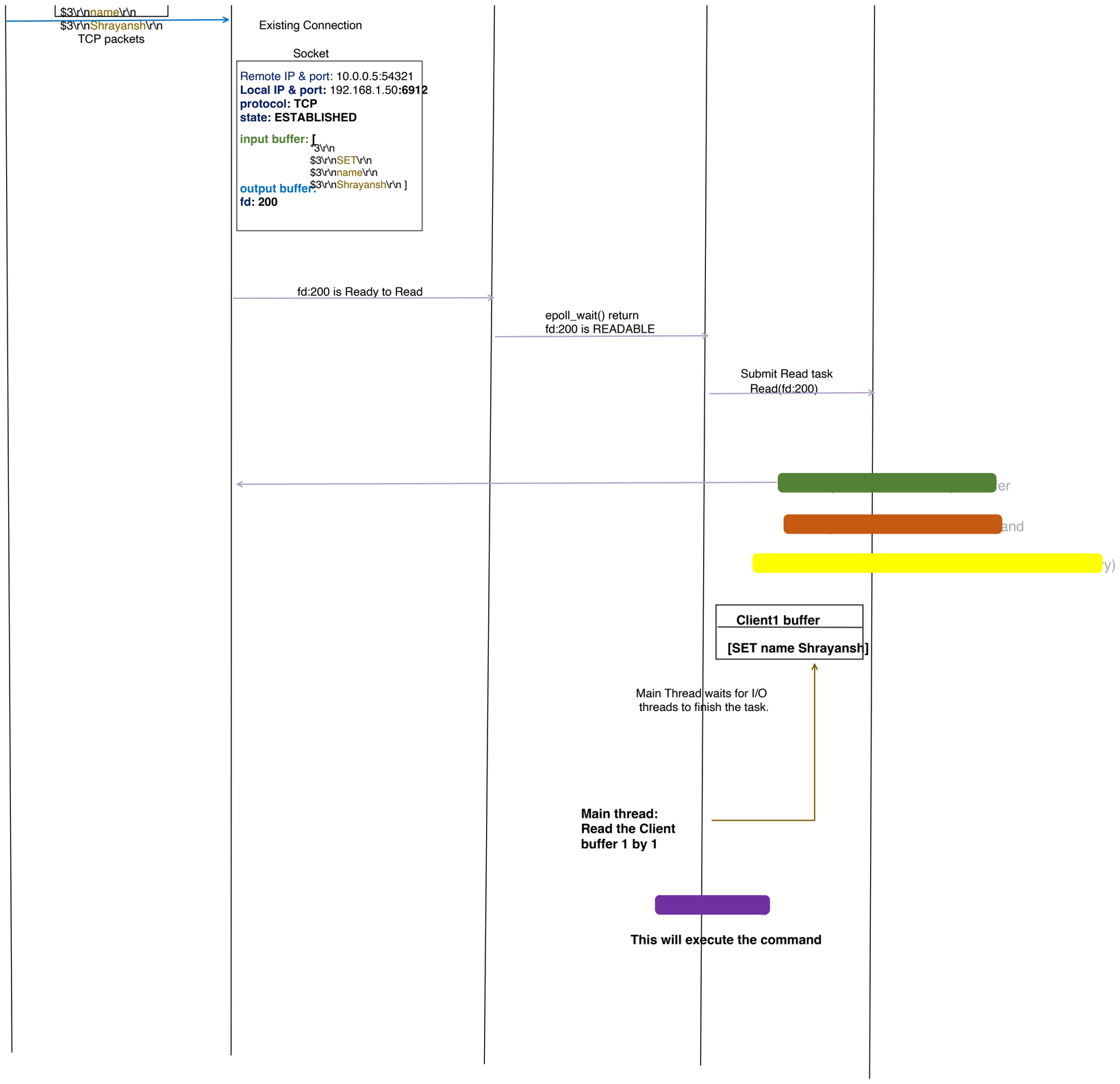
Lets say with RESP encoding

*3\r\n$3\r\nSET\r\n$3\r\nname\r\n$3\r\nShrayansh\r\n



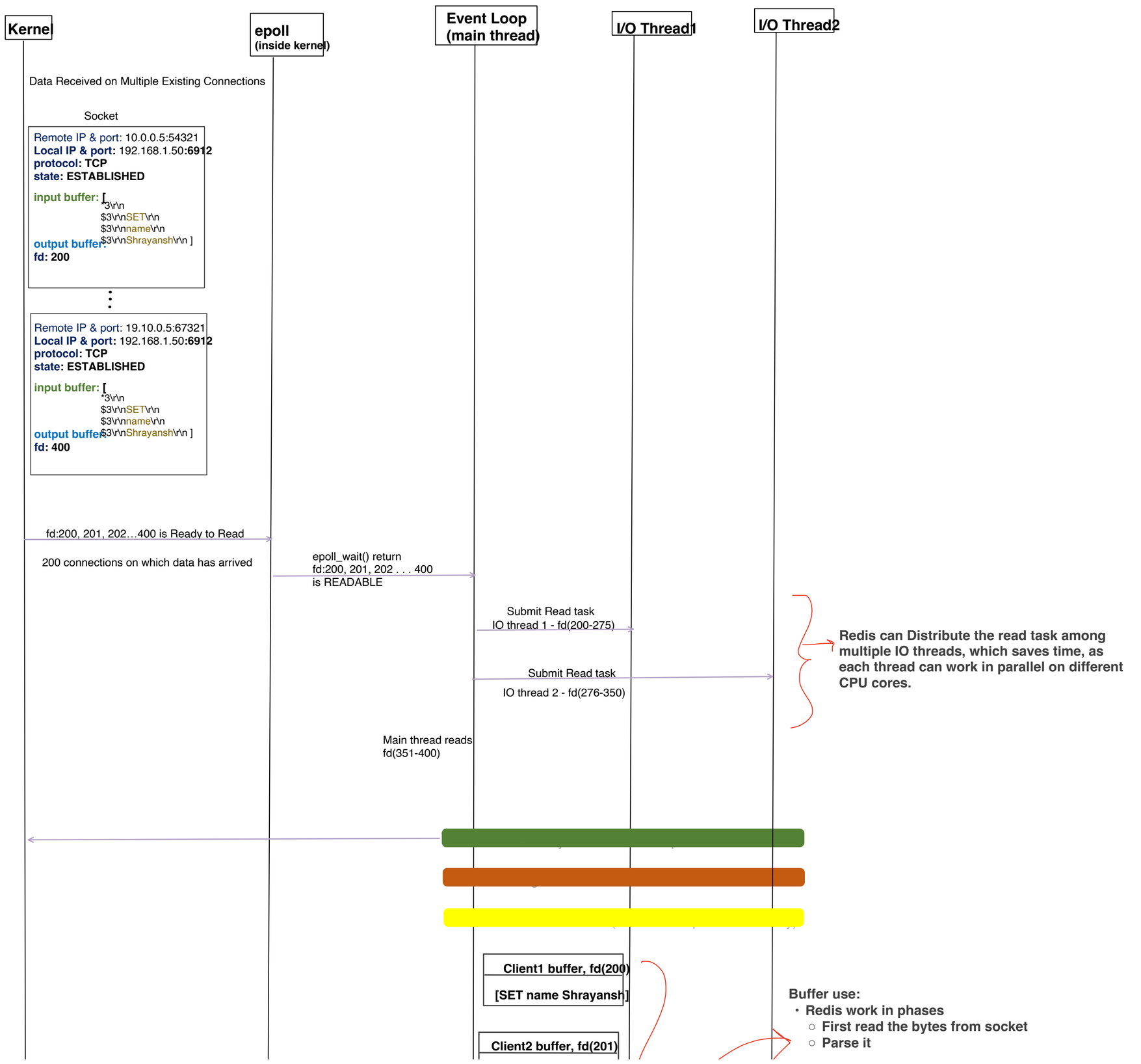
4. **Client (Springboot app) sends data to Redis Server**

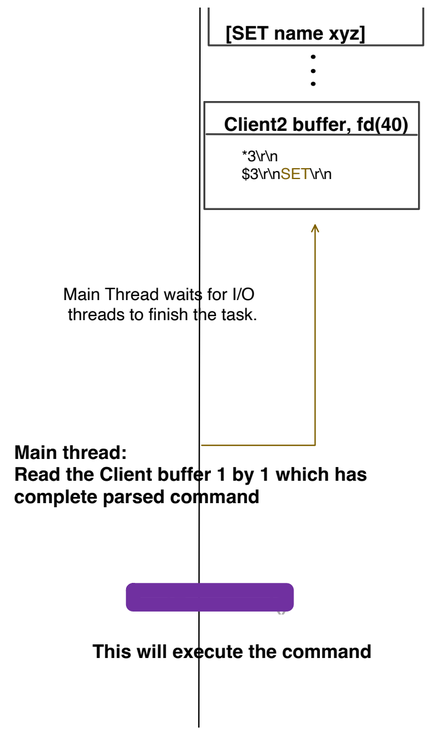




Doubts in this, before we proceed, what happens during execution:

Doubt1: What's the real benefit of I/O thread?
Doubt2: Why we have Client Buffer? Say if we have 1000 clients, then what's the point of maintaining buffer for each client?





- Then move to execution phase

And reading of bytes happens in parallel from multiple connections, while execution happens only 1 by 1.

So Client wise buffer is required.

- **When for any connection, partial command is only received (multiple TCP packets scenario),**
  That partial RESP data can wait in client buffer and when rest of the data arrived, its merged and then parsed into decoded command.

5. **<u>What happens when Redis 'main thread' start execution of the command</u>**

So when Redis run the command:

SET name Shrayansh

How it actually store?

1. **Hash Table is maintained in RAM (called main dictionary)**

Key = hash(name) = K1

Key	Value

K1	redisObject {
	type: what kind of value: STRING, LIST, HASH, SET, ZSET, ...
	encoding: HOW this value is physically stored internally
	ptr: pointer to the actual data
	}

Example:

HSET user:101 name "RJ" age "30" city "Pune"

```
redisObject: {
        type: HASH
        encoding: listpack
        ptr: 1000 and size 80bytes
    }
```

Encoding is totally internal of Redis, say it see that values:
Name -> RJ
Age -> 30
City -> Pune
It's a small hash, than it can store it as listpack, instead of hash table.

Name	RJ	Age	30	City	Pune	

Start-1000 End-1080

If hash grows large, it can switch to different internal representation, whereas type remains HASH only encoding type differs.

Interviewer sometimes ask:

Since the main dictionary is using Hash Table internally, then how it maintains O(1) lookup

What they mean to say:

hash(name) = K1
hash(age) = K1
.
.
All key come is K1 only

Key	Value
K1	redisObject1 redisObject2 redisObject3 . . . redisObjectN

Then Lookup complexity is O(N)

Hash-Flooding DoS Attacks (HashDoS). An attacker who knew the hashing algorithm could purposely craft hundreds of different keys that all hashed to the **exact same hash table bucket** (causing worst-case O(N)).

That's why Redis calls average lookup time is O(1)

But REDIS try its best to avoid above use-case by:

- **Choosing strong hash function (SipHash)**
- **Rehashing (when data grows, increasing the size of Hash table also helps to reduce collisions)**

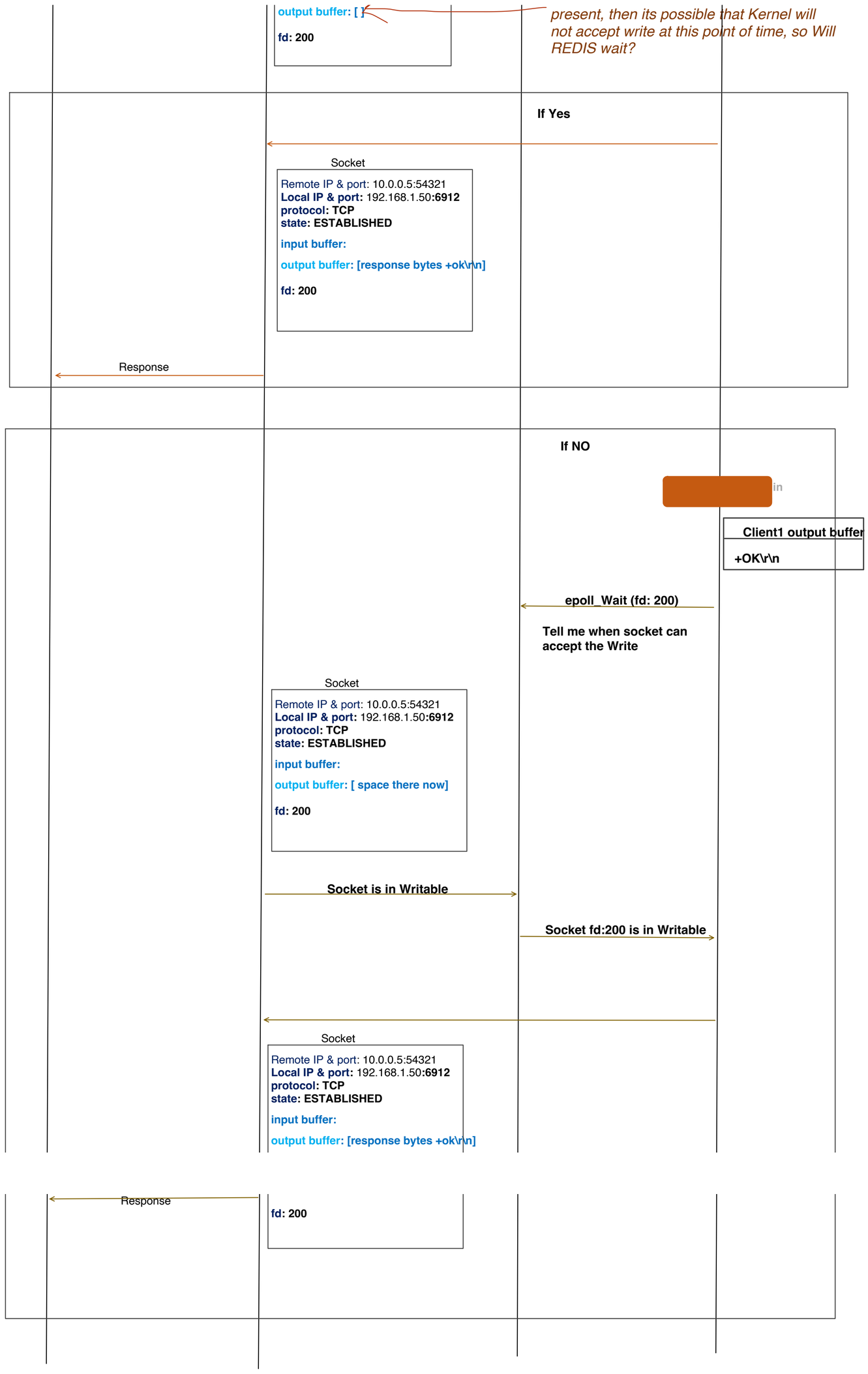
But again question comes, Does Redis stops while it does the Rehashing because all existing keys need to be Rehashed.

Answer is NO.

1. Redis creates a second, bigger, empty table (roughly double the size) alongside the current one.
   Internally, these are called say: ht[0] (the old table) and ht[1] (the new one).
2. Instead of moving everything at once, Redis moves small no of entries from ht[0] to ht[1] every time it processes a command that touch that hash table
3. During this transition period, both tables exist at once, a lookup first checks ht[0], and if not found there, checks ht[1].
4. Once every bucket has been migrated, ht[0] is thrown away, and ht[1] becomes the new main table.

But yes, true worst case is O(N)

6. <u>Response</u>



Now, Redis will again go back to:

- Look at its pending Client buffer, if any command is pending
  If no
- goes to epoll_Wait() state

7. Persistence

Redis persistence technique:

- **RDB (Redis Database)**
- **AOF (Append only file)**

The problem: saving data to disk takes time, and shouldn't freeze the main thread

So how Redis handle this?

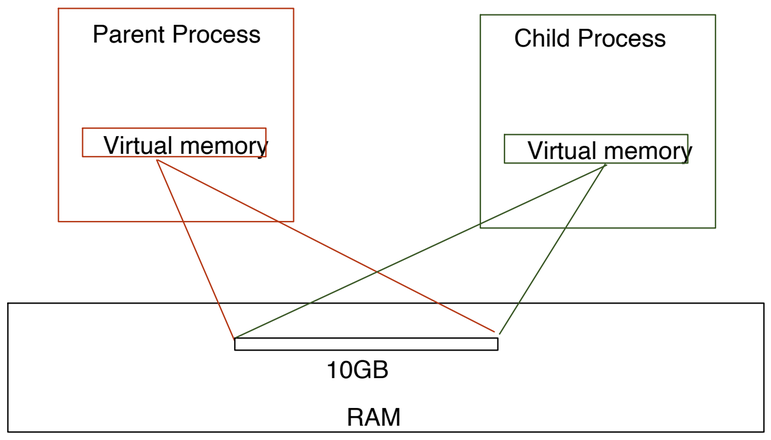
Answser: fork()

By creating a total new child process

But, creating a child process from parent process, is it not time consuming, it has to copy all the memory data (say 10GB)

Answser: NO, Redis follows Copy-on-Write approach

Means, the child process only meant to Read the memory and write to Disk, so initially child Process also point to same memory.



Note: we will go in deep when we will cover AOF / RDB topic.